

# Trabajo Práctico Final

## Sistema de Soporte para Compañía de Videojuegos

### Algoritmos y Estructuras de Datos II

**Docente: Matías Ruiz**

**Alumnos: Luciana Micaela Flores - Matias Ezequiel Gonzalez**

### Requerimiento del problema

Las compañías de videojuegos reciben a diario solicitudes de soporte relacionadas con la seguridad y gestión de cuentas, además de pagos o compras dentro del juego. Algunos ejemplos son cuentas hackeadas, cambios de región o contraseña, reembolsos, problemas con tarjetas de regalo y contracargos.

El sistema debe permitir registrar cada solicitud, conservarla para consultas posteriores y resolverla desde un panel de soporte. La regla principal es que los casos más urgentes deben atenderse antes, aunque hayan llegado después. También se diferencia entre jugadores, que crean o consultan sus propios tickets, y personal de soporte, que los atiende.

### Objetivo del sistema

Desarrollar una aplicación de consola en Java que registre, organice y gestione tickets de soporte. El cliente puede crear un ticket, ver los tickets vinculados al usuario con el que ingresó o buscar uno propio por su número de identificación. El staff inicia sesión con cuentas predefinidas, puede atender el siguiente ticket prioritario, registrar una resolución y consultar tickets no resueltos, resueltos o uno puntual por ID.

En esta versión de consola, el nombre de usuario ingresado representa una identidad previamente validada por la plataforma del videojuego. En un sistema real, la plataforma enviaría ese usuario al módulo de soporte. De esta manera, el sistema no guarda contraseñas de jugadores y el cliente solo accede a los tickets asociados a su cuenta.

### Niveles de prioridad

| Prioridad | Nivel | Tipos de solicitud                                                  |
|-----------|-------|---------------------------------------------------------------------|
| 1         | Alta  | Me hackearon la cuenta; Resolución de contracargos                  |
| 2         | Media | Reembolsar compra dentro del juego; Problemas con tarjeta de regalo |
| 3         | Baja  | Cambiar la región de la cuenta; Solicitud de cambio de contraseña   |

### Análisis del problema

El problema no puede resolverse con una cola común, porque una cola común atiende estrictamente por orden de llegada. En este caso, un ticket por cuenta hackeada no puede

quedar detrás de una consulta menos urgente solo porque se registró antes. Por eso se necesita una cola con prioridad: primero se considera la urgencia y, cuando dos tickets tienen la misma prioridad, se respeta el orden en que llegaron.

Además de atender los pendientes, el sistema debe conservar todos los tickets, incluso los resueltos. Debe poder encontrar un ticket rápido por su ID, recuperar los tickets de un jugador y comprobar que un ID aleatorio no esté repetido. Ninguna estructura cubre todas estas necesidades por sí sola, por lo que se combinan una cola con prioridad y diccionarios.

## Estructuras de datos utilizadas

La solución combina estructuras con responsabilidades distintas. La cola organiza la atención; los diccionarios permiten buscar; y otras clases de Java ayudan a modelar o persistir los datos.

| Estructura                  | Rol en el sistema                                                                  | Implementación utilizada                                                     |
|-----------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Cola con prioridad          | Mantiene solo los tickets pendientes y entrega el que corresponde atender primero. | IColaPrioridad / ColaPrioridad implementada con PriorityQueue (heap binario) |
| Diccionario por ID          | Permite recuperar un ticket puntual y verificar que un ID no se repita.            | Map<Integer, Ticket> con HashMap                                             |
| Diccionario por usuario     | Relaciona a cada jugador con el historial de sus tickets.                          | Map<String, List<Ticket>> con HashMap y ArrayList                            |
| Diccionario de credenciales | Valida el acceso de las cuentas de staff definidas para el TP.                     | Map<String, String> con HashMap                                              |
| Catálogo de tipos           | Define los tipos de solicitud, el área y la prioridad asignada a cada uno.         | enum HerramientaSoporte                                                      |
| Generador de ID             | Genera un número aleatorio de cinco cifras y comprueba que esté disponible.        | Random + diccionario por ID                                                  |

## Por qué elegimos estas estructuras

La cola con prioridad resuelve el problema más importante: saber qué ticket atender primero. La cola guarda los pendientes ordenados por prioridad. En esta implementación, el ticket que debe salir primero queda siempre en una posición conocida del arreglo, por lo que el staff no necesita recorrer todos los reclamos para decidir cuál sigue.

Los diccionarios se usan para las búsquedas. El diccionario por ID permite llegar a un ticket concreto sin revisar todos los registrados. El diccionario por usuario permite recuperar solo el historial del jugador actual, lo que además ayuda a que el cliente no vea tickets de otra persona. Estas dos búsquedas no las podría resolver bien una cola con prioridad.

Las estructuras se complementan: la cola sabe cuál es el siguiente ticket a atender, pero no sirve para buscar por ID. El diccionario busca rápido por ID o usuario, pero no mantiene el

orden de atención. Usar ambas evita forzar a una sola estructura a hacer tareas para las que no fue elegida.

## Implementación de la cola con prioridad

El TDA Cola con Prioridad está definido por la interfaz `IColaPrioridad`, que indica qué operaciones existen: acolar, ver el próximo ticket, desacolar, etc. La implementación concreta usa la clase `PriorityQueue` de Java, que por dentro es un heap binario. Usar `PriorityQueue` no le quita el carácter de TDA: el TDA es el contrato (la interfaz) y el heap es una de las formas válidas de cumplirlo.

El criterio de orden no está fijo dentro de la cola: se le pasa un comparador (`ComparadorPrioridad`) que ordena primero por prioridad y, ante el mismo nivel, por orden de llegada. Así, cuando entra un ticket, el heap lo ubica en su lugar reacomodándose internamente, y el ticket más urgente queda siempre listo para salir. Si dos solicitudes tienen la misma prioridad, se atiende primero la que llegó antes.

Elegimos `PriorityQueue` porque es eficiente —acola y desacola en  $O(\log n)$ — y porque, al estar detrás de la interfaz, no ata el resto del sistema a esa clase. Si en el futuro quisiéramos otra implementación del mismo TDA (por ejemplo una lista ordenada), bastaría con cambiar esa clase sin tocar el gestor.

## Criterios de diseño (los cuatro pilares)

### 1 Programar contra interfaces

Programar contra interfaces significa que la parte principal del sistema trabaja con las operaciones que necesita, sin depender de los detalles internos de una clase. En otras palabras: primero se define qué debe poder hacer la estructura y después se decide cómo se guardan los datos.

En el proyecto, el gestor de tickets necesita agregar un ticket pendiente, consultar cuál es el siguiente y retirarlo cuando se resuelve. Esas operaciones están definidas por `IColaPrioridad`. El gestor usa ese contrato y no necesita saber en qué posición del arreglo se guarda cada ticket ni cómo se realizan los corrimientos.

También se utilizan los tipos generales `Map` y `List` para los diccionarios y los historiales. Después se eligen `HashMap` y `ArrayList` como implementaciones concretas porque son útiles para buscar por clave y guardar varios tickets por usuario. Así, la lógica del sistema queda más ordenada: el gestor se concentra en registrar, buscar y atender, mientras cada estructura se ocupa de guardar los datos.

### 2 Lo dinámico no siempre es superior

En este proyecto no todas las estructuras se eligen iguales. Lo que puede variar de tamaño se resuelve con estructuras dinámicas: la cola con prioridad, que crece con la cantidad de pendientes, los diccionarios y los historiales de cada usuario. En cambio, lo que es fijo y conocido de antemano se resuelve con estructuras estáticas.

Los tipos de solicitud son un buen ejemplo de lo estático: son seis y no cambian, así que se representan con un enum (`HerramientaSoporte`) y no con una lista que pueda crecer. Lo mismo ocurre con los tres niveles de prioridad, que son constantes. Para esos datos, una estructura dinámica no aportaría nada y solo agregaría complejidad.

Los diccionarios, los historiales y la propia cola sí pueden crecer según la cantidad de tickets y de usuarios, que no se conoce de antemano. Por eso usan estructuras dinámicas, que se ajustan al volumen real sin fijar un tope por adelantado.

La idea no es que lo dinámico sea siempre mejor. Se elige según el problema: estructuras estáticas para lo que es fijo y acotado (los tipos de solicitud y las prioridades) y dinámicas para lo que puede variar (la cola de pendientes, los historiales y los diccionarios).

### 3 Corrimiento, tiempo y espacio

El corrimiento es el costo de desplazar elementos: en un arreglo ordenado, insertar un ticket en el medio obliga a correr una posición todos los que vienen después, lo que cuesta  $O(n)$ . Para evitar ese costo, la cola con prioridad no usa un arreglo ordenado sino un heap: al acolar, el heap se reacomoda internamente en  $O(\log n)$ , sin desplazar todos los elementos.

Consultar el próximo ticket es directo: el más urgente está siempre en la raíz del heap, así que verlo cuesta  $O(1)$ . Atenderlo, es decir sacarlo, también es eficiente: el heap quita la raíz y se reacomoda en  $O(\log n)$ . De esta forma, tanto encolar como atender se mantienen en  $O(\log n)$  por más que crezca la cantidad de pendientes.

La siguiente tabla diferencia el costo interno de cada estructura del costo total de una acción. Por ejemplo, atender el ticket dentro de la cola es  $O(\log n)$ , pero el sistema luego guarda los cambios en un archivo JSON; como vuelve a escribir todos los tickets, la operación completa termina siendo  $O(n)$ .

| Operación                   | Estructura                 | Costo           | Explicación                                                                                                 |
|-----------------------------|----------------------------|-----------------|-------------------------------------------------------------------------------------------------------------|
| Acolar ticket pendiente     | ColaPrioridad (heap)       | $O(\log n)$     | Inserta en el heap, que se reacomoda; no recorre ni desplaza todos los tickets.                             |
| Ver próximo ticket          | ColaPrioridad (heap)       | $O(1)$          | El ticket más urgente está siempre en la raíz del heap.                                                     |
| Desacolar ticket            | ColaPrioridad (heap)       | $O(\log n)$     | Quita la raíz (el más urgente) y reacomoda el heap.                                                         |
| Crear ticket completo       | Cola + diccionarios + JSON | $O(n)$          | Encolar es $O(\log n)$ , pero el guardado en JSON reescribe todos los tickets, por eso el total es $O(n)$ . |
| Atender y resolver completo | Cola + JSON                | $O(n)$          | Desacolar es $O(\log n)$ , pero guardar todos los tickets en JSON es $O(n)$ .                               |
| Buscar ticket por ID        | HashMap                    | $O(1)$ promedio | Se consulta directamente la clave del ticket.                                                               |

| Operación                       | Estructura          | Costo           | Explicación                                                                              |
|---------------------------------|---------------------|-----------------|------------------------------------------------------------------------------------------|
| Buscar tickets de un usuario    | HashMap + ArrayList | $O(1) + O(k)$   | Se obtiene la lista del usuario y se recorren solo sus $k$ tickets para mostrarlos.      |
| Agregar ticket al historial     | ArrayList           | $O(1)$ amort.   | Normalmente se agrega al final; solo algunas veces debe ampliar su arreglo interno.      |
| Listar resueltos o no resueltos | Todos los tickets   | $O(n)$          | Se revisa cada ticket para verificar su estado.                                          |
| Generar y validar ID            | Random + HashMap    | $O(1)$ esperado | Genera un número de cinco cifras y consulta si ya existe; repite solo ante una colisión. |

En el historial aparece  $O(1)$  amortizado. Esto significa que casi todas las inserciones al final son constantes, pero cada tanto ArrayList debe ampliar su arreglo interno y copiar sus elementos. Al considerar muchas inserciones juntas, el costo promedio se mantiene en  $O(1)$ .

### Consideraciones sobre eficiencia y espacio

El costo total de memoria del sistema es  $O(n)$ , porque a medida que se registran tickets también crecen los diccionarios y los historiales. Cada ticket existe como un objeto y las demás estructuras guardan referencias a ese mismo ticket, no copias independientes. La cola también crece con la cantidad de pendientes, pero como guarda referencias a los mismos tickets, no duplica la información.

Los costos se expresan con Big O para mostrar cómo crece el trabajo cuando aumenta la cantidad de datos, no para medir milisegundos reales. En este proyecto, las operaciones más frecuentes se mantienen baratas: encolar y atender son  $O(\log n)$  y las búsquedas por ID son  $O(1)$  promedio. El  $O(n)$  del alta completa no viene de la cola, sino del guardado en JSON, que reescribe todo el archivo.

## 4 Elegir según el problema

No se eligió una estructura solo porque tenga el menor Big O en todas las operaciones. Se eligió considerando qué necesita el sistema y qué temas se busca aplicar. La cola con prioridad permite respetar la regla del negocio: urgencia primero y llegada como desempate. Los diccionarios permiten las búsquedas que la cola no puede resolver.

También se compararon alternativas. Algunas no respetan la urgencia, otras obligan a recorrer todos los tickets y otras modelan problemas distintos. La tabla muestra por qué no resultan adecuadas para este sistema.

| Alternativa | Por qué no se eligió                                                                                                             |
|-------------|----------------------------------------------------------------------------------------------------------------------------------|
| if / else   | Sirve para tomar decisiones puntuales, pero no permite almacenar ni ordenar una cantidad variable de tickets según su prioridad. |

| Alternativa                 | Por qué no se eligió                                                                                                                           |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| switch                      | Sirve para elegir entre opciones fijas del menú o tipos de reclamo, pero no permite guardar tickets ni determinar cuál debe atenderse primero. |
| Arreglo o lista sin ordenar | Permite guardar tickets, pero para encontrar el más urgente habría que recorrerlos todos: $O(n)$ en cada atención.                             |
| Cola común (FIFO)           | Respetar la llegada, pero ignora la urgencia. Un caso grave podría esperar detrás de un reclamo leve.                                          |
| Pila (LIFO)                 | Atendería primero el último ticket registrado, lo que no coincide con ninguna regla del sistema.                                               |
| Conjunto                    | No mantiene un criterio de atención ni indica qué ticket debe salir primero.                                                                   |
| Árbol binario de búsqueda   | Podría ordenar por una clave compuesta, pero agrega complejidad que no se necesita para este alcance.                                          |
| Grafo                       | Sirve para representar relaciones entre entidades. Los tickets no forman una red de relaciones entre sí.                                       |

## Fundamentos de Java utilizados

Además de los TDAs, el proyecto utiliza elementos básicos de Java para separar responsabilidades y permitir que el usuario interactúe por consola. Estas herramientas no reemplazan a las estructuras de datos: las acompañan para que el sistema pueda funcionar y mostrar resultados.

| Elemento de Java | Cómo se usa en el proyecto                                                                                                                                                                 |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Clases y objetos | Ticket representa cada solicitud; GestorTickets concentra la lógica; Menu organiza la interacción por consola; ColaPrioridad implementa el TDA.                                            |
| Encapsulamiento  | Los atributos de Ticket son privados y se consultan o modifican mediante métodos, evitando cambios directos desde cualquier parte del programa.                                            |
| enum             | HerramientaSoporte define los tipos válidos de solicitud junto con su área y prioridad.                                                                                                    |
| Scanner          | Se usa en Menu para leer opciones, nombre de usuario, credenciales, descripciones e IDs ingresados por teclado. También permite validar que la opción elegida pertenezca al menú mostrado. |
| Random           | Genera el número de identificación aleatorio de cinco cifras antes de registrar cada ticket.                                                                                               |
| Archivos JSON    | Permiten conservar tickets entre ejecuciones. Al iniciar se cargan; al crear o resolver se guardan nuevamente.                                                                             |

## Temas de la cursada aplicados

El trabajo utiliza más de cuatro temas de la lista propuesta. Los principales son los siguientes:

| Tema                                        | Aplicación en el sistema                                                                                                                 |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Introducción a los TDA                      | Se pensó primero qué operaciones necesita el sistema y después cómo implementarlas.                                                      |
| Especificación de TDA y notación            | IColaPrioridad define las operaciones de la cola sin mostrar sus detalles internos.                                                      |
| Fundamentos de Java                         | Se utilizan clases, objetos, enum, encapsulamiento, Scanner, Random, condicionales, ciclos y archivos JSON.                              |
| Implementaciones dinámicas                  | La cola con prioridad (heap), los diccionarios y los historiales crecen y decrecen en ejecución según la cantidad de tickets y usuarios. |
| TDA Cola con Prioridad                      | Es el núcleo del sistema: prioriza la urgencia y mantiene el orden de llegada cuando las prioridades coinciden.                          |
| TDA Diccionario                             | Los diccionarios permiten buscar por ID, relacionar usuarios con tickets y validar credenciales del staff.                               |
| Resolución de problemas combinando TDAs     | La cola se ocupa de la atención y los diccionarios se ocupan de las búsquedas y relaciones.                                              |
| Eficiencia, complejidad espacial y temporal | Se analizan los costos $O(\log n)$ , $O(1)$ y $O(n)$ , el corrimiento que evita el heap, la memoria y el impacto del guardado en JSON.   |
| Comparación de implementaciones             | Se compara la solución elegida con un arreglo o lista sin ordenar, cola común, pila, conjunto, ABB y grafo.                              |

## Casos de uso y pruebas

Los siguientes casos permiten comprobar que las reglas principales del sistema se cumplan:

| Caso                           | Pasos                                                                                | Resultado esperado                                                                  |
|--------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Crear ticket de prioridad alta | Cliente identificado -> Crear ticket -> "Me hackearon la cuenta" + descripción.      | Se registra con ID aleatorio único de cinco cifras, prioridad 1 y estado Pendiente. |
| Orden por prioridad            | Crear un ticket de prioridad baja y luego uno de prioridad alta -> staff -> Atender. | Sale primero el ticket de prioridad alta, aunque haya llegado después.              |
| Desempate por llegada          | Crear dos tickets de la misma prioridad -> staff -> Atender dos veces.               | Sale primero el ticket registrado antes.                                            |

| Caso                           | Pasos                                                                                      | Resultado esperado                                                                         |
|--------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Consultar por ID como cliente  | Cliente -> Consultar tickets -> Buscar por ID -> ingresar el número de uno de sus tickets. | Muestra tipo, estado y resolución solo si el ticket pertenece al usuario actual.           |
| Visualizar tickets del cliente | Cliente -> Consultar tickets -> Visualizar tickets registrados.                            | Muestra solo los tickets asociados al usuario actual. Si no existen, informa la situación. |
| Login de staff                 | Ingresar como staff con credenciales correctas e incorrectas.                              | Con credenciales correctas entra al panel; con credenciales incorrectas muestra un error.  |
| Atender y resolver             | Staff -> Atender ticket -> ingresar resolución.                                            | El ticket pasa de Pendiente a Resuelto, sale de la cola y sigue disponible para consultas. |
| Consultar tickets como staff   | Staff -> Consultar tickets -> No resueltos / Resueltos / Buscar por ID.                    | Muestra solo el listado o ticket solicitado según la opción seleccionada.                  |
| Generar ID aleatorio           | Crear varios tickets consecutivos.                                                         | Cada ticket recibe un ID de cinco cifras que no coincide con otro ticket registrado.       |

## Posibilidad de escalabilidad

La separación entre el menú, el gestor, los tickets y las estructuras de datos permite ampliar el sistema sin mezclar responsabilidades. Por ejemplo, podrían agregarse más técnicos, nuevas categorías, comentarios internos, historial de cambios de estado, métricas o notificaciones.

La cola con prioridad crece de forma dinámica y mantiene sus operaciones en  $O(\log n)$  aunque aumente la cantidad de reclamos, así que acompaña bien el crecimiento. Y como el gestor trabaja contra la interfaz `IColaPrioridad`, si alguna vez conviniera otra implementación del TDA, el cambio quedaría concentrado en esa clase y no en todo el sistema.

## Integración con la plataforma del videojuego

En una versión real, las cuentas de jugadores se validarían en la plataforma del juego o en la web de la empresa. El módulo de soporte recibiría el usuario ya autenticado y no tendría que almacenar contraseñas. El prototipo simula ese comportamiento pidiendo el nombre de usuario al iniciar como cliente. Por eso el jugador puede ver solamente los tickets asociados a su identidad.

## Generación de identificadores

Cada ticket recibe un número aleatorio entre 10000 y 99999. Antes de usarlo, el sistema verifica en el diccionario por ID que ese número no esté asignado. Esta estrategia es suficiente para el TP porque hay 90.000 valores posibles. En una aplicación con mucho volumen, sería preferible que una base de datos genere el ID o utilizar un UUID.

## Limitaciones actuales

El filtrado de tickets por estado revisa todos los tickets registrados, por lo que cuesta  $O(n)$ . Además, al guardar en JSON se reescribe el archivo completo, lo que también cuesta  $O(n)$ . Son decisiones aceptables para una aplicación de consola pequeña, pero en producción se podrían usar una base de datos y colecciones adicionales por estado.

## Conclusión

El sistema resuelve el requerimiento combinando una cola con prioridad para organizar la atención y diccionarios para realizar búsquedas. La cola se implementó con `PriorityQueue` (un heap) detrás de la interfaz `IColaPrioridad`, respetando la prioridad y, ante empate, el orden de llegada, con un costo de  $O(\log n)$  al encolar y atender. Los diccionarios complementan esa cola al permitir buscar por ID, recuperar el historial de cada usuario y validar que los IDs no se repitan.

Usar `PriorityQueue` no le quita al trabajo el carácter de TDA: lo importante es que el sistema depende de la interfaz `IColaPrioridad` (el contrato) y no de la clase concreta, de modo que la implementación podría cambiarse sin afectar al resto. A la vez, el trabajo incorpora fundamentos de Java como `Scanner`, `enum`, clases, encapsulamiento, `Random` y persistencia JSON. Así, las estructuras y decisiones técnicas responden al problema planteado y a los temas trabajados durante la cursada.